LAPORAN PRAKTIKUM

- **Identitas Praktikum**

  Nama MK : Praktikum Pemrograman Berorientasi Objek

  Kode MK : PBO

  Bobot SKS : 3 SKS

  Tempat : L – MM

  Hari, tanggal : Jumat, 24-4-2026

  Jam : 12.30 – 15.30

  Topik praktikum : Modul-7

- **Identitas Mahasiswa**

  Nama lengkap : Denna Wahyu Setyobudi

  NIM : 103112430206

  Program Studi : S-1 Teknik Informatika

- **Algoritma / studi kasus (jika ada) & Penjelasan**

  Program ini dirancang untuk mensimulasikan sistem pembayaran e-commerce ShopKita menggunakan konsep Pemrograman Berorientasi Objek (PBO), yaitu abstraksi dan *interface*. *Interface* Pembayaran menetapkan standar aksi yang harus dimiliki setiap metode, seperti meProgram ini dirancang untuk mensimulasikan sistem pembayaran e-commerce ShopKita menggunakan konsep Pemrograman Berorientasi Objek (PBO), yaitu abstraksi dan *interface*. *Interface* Pembayaran menetapkan standar aksi yang harus dimiliki setiap metode, seperti memproses pembayaran dan mengecek saldo, sementara *abstract class* MetodePembayaran berfungsi sebagai kerangka dasar untuk menyimpan riwayat transaksi dan mengelola identitas metode secara terpusat. Dengan struktur ini, kode menjadi lebih rapi dan mudah dikembangkan jika di masa depan terdapat penambahan metode pembayaran baru.

  Setiap kelas turunan, yaitu DompetDigital, KartuKredit, dan RekeningBank, mengimplementasikan logika validasi yang berbeda-beda sesuai dengan aturan bisnis yang ditetapkan. Misalnya, RekeningBank secara otomatis menyertakan biaya administrasi sebesar Rp 2.500 dalam setiap pengecekan saldo dan transaksi, sedangkan KartuKredit berfokus pada batas limit penggunaan. Melalui kelas Main, program menunjukkan kemampuannya dalam menangani berbagai skenario, termasuk memberikan laporan detil saat sebuah transaksi berhasil dilakukan atau ditolak karena tidak memenuhi kriteria validasi.

  mproses pembayaran dan mengecek saldo, sementara *abstract class* MetodePembayaran berfungsi sebagai kerangka dasar untuk menyimpan riwayat transaksi dan mengelola identitas metode secara terpusat. Dengan struktur ini, kode menjadi lebih rapi dan mudah dikembangkan jika di masa depan terdapat penambahan metode pembayaran baru.

Setiap kelas turunan, yaitu DompetDigital, KartuKredit, dan RekeningBank, mengimplementasikan logika validasi yang berbeda-beda sesuai dengan aturan bisnis yang ditetapkan. Misalnya, RekeningBank secara otomatis menyertakan biaya administrasi sebesar Rp 2.500 dalam setiap pengecekan saldo dan transaksi, sedangkan KartuKredit berfokus pada batas limit penggunaan. Melalui kelas Main, program menunjukkan kemampuannya dalam menangani berbagai skenario, termasuk memberikan laporan detil saat sebuah transaksi berhasil dilakukan atau ditolak karena tidak memenuhi kriteria validasi.

- **Kode Program & Penjelasan**

Pembayaran.java

```java
public interface Pembayaran {
    void bayar(double nominal);
    double cekSaldo();
    void getRiwayat();
}
```

MetodePembayaran.java

```java
import java.util.ArrayList;
import java.util.List;

public abstract class MetodePembayaran implements Pembayaran {
    protected String namaMetode;
    protected List<String> riwayatTransaksi;

    public MetodePembayaran(String namaMetode) {
        this.namaMetode = namaMetode;
        this.riwayatTransaksi = new ArrayList<>();
    }

    protected abstract boolean validasiPembayaran(double nominal);

    @Override
    public void getRiwayat() {
        System.out.println("Riwayat Transaksi - " + namaMetode + ":");
        if (riwayatTransaksi.isEmpty()) {
            System.out.println("- Belum ada transaksi.");
        } else {
            for (String riwayat : riwayatTransaksi) {
                System.out.println("- " + riwayat);
            }
        }
        System.out.println();
    }

    protected void tambahRiwayat(String pesan) {
        riwayatTransaksi.add(pesan);
    }
}
```

DompetDigital.java

```java
public class DompetDigital extends MetodePembayaran {
    private double saldo;

```

```java
    public DompetDigital(String namaMetode, double saldoAwal) {
        super(namaMetode);
        this.saldo = saldoAwal;
    }

    @Override
    protected boolean validasiPembayaran(double nominal) {
        return saldo >= nominal;
    }

    @Override
    public void bayar(double nominal) {
        if (validasiPembayaran(nominal)) {
            saldo -= nominal;
            tambahRiwayat("Berhasil: Bayar Rp" + nominal + " - Sisa Saldo: Rp" +
saldo);
            System.out.println(namaMetode + " - Pembayaran Rp" + nominal + "
BERHASIL.");
        } else {
            tambahRiwayat("Gagal: Bayar Rp" + nominal + " - Saldo tidak
mencukupi.");
            System.out.println(namaMetode + " - Pembayaran Rp" + nominal + "
GAGAL.");
        }
    }

    @Override
    public double cekSaldo() {
        return saldo;
    }
}
```

KartuKredit.java

```java
public class KartuKredit extends MetodePembayaran {
    private double tagihan;
    private double limitKredit;

    public KartuKredit(String namaMetode, double limitKredit) {
        super(namaMetode);
        this.limitKredit = limitKredit;
        this.tagihan = 0;
    }

    @Override
    protected boolean validasiPembayaran(double nominal) {
        return (tagihan + nominal) <= limitKredit;
    }

    @Override
    public void bayar(double nominal) {
        if (validasiPembayaran(nominal)) {
            tagihan += nominal;
            tambahRiwayat("Berhasil: Bayar Rp" + nominal + " - Total Tagihan: Rp"
+ tagihan);
```

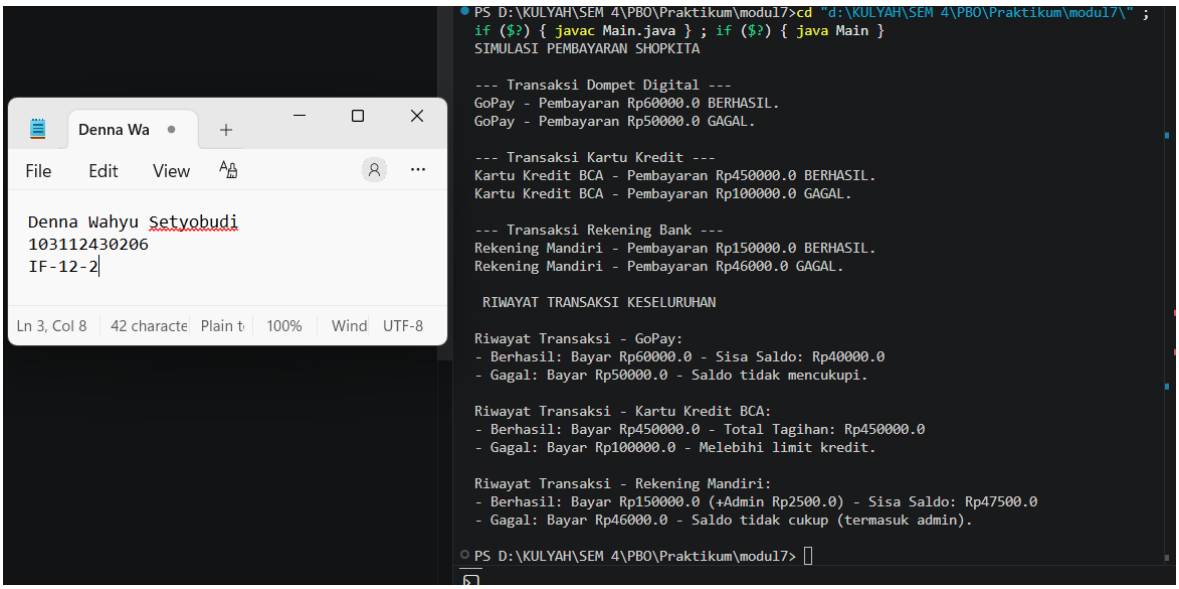
```
            System.out.println(namaMetode + " - Pembayaran Rp" + nominal + " BERHASIL.");
        } else {
            tambahRiwayat("Gagal: Bayar Rp" + nominal + " - Melebihi limit kredit.");
            System.out.println(namaMetode + " - Pembayaran Rp" + nominal + " GAGAL.");
        }
    }

    @Override
    public double cekSaldo() {
        return limitKredit - tagihan;
    }
}
```

RekeningBank.java

```
public class RekeningBank extends MetodePembayaran {
    private double saldo;
    private final double BIAYA_ADMIN = 2500;

    public RekeningBank(String namaMetode, double saldoAwal) {
        super(namaMetode);
        this.saldo = saldoAwal;
    }

    @Override
    protected boolean validasiPembayaran(double nominal) {
        return saldo >= (nominal + BIAYA_ADMIN);
    }

    @Override
    public void bayar(double nominal) {
        if (validasiPembayaran(nominal)) {
            saldo -= (nominal + BIAYA_ADMIN);
            tambahRiwayat("Berhasil: Bayar Rp" + nominal + " (+Admin Rp" + BIAYA_ADMIN + ") - Sisa Saldo: Rp" + saldo);
            System.out.println(namaMetode + " - Pembayaran Rp" + nominal + " BERHASIL.");
        } else {
            tambahRiwayat("Gagal: Bayar Rp" + nominal + " - Saldo tidak cukup (termasuk admin).");
            System.out.println(namaMetode + " - Pembayaran Rp" + nominal + " GAGAL.");
        }
    }

    @Override
    public double cekSaldo() {
        return saldo;
    }
}
```

Main.java

```
public class Main {
    public static void main(String[] args) {
```

```java
        System.out.println("SIMULASI PEMBAYARAN SHOPKITA \n");

        Pembayaran gopay = new DompetDigital("GoPay", 100000);
        Pembayaran kartuKredit = new KartuKredit("Kartu Kredit BCA", 500000);
        Pembayaran mandiri = new RekeningBank("Rekening Mandiri", 200000);

        System.out.println("--- Transaksi Dompet Digital ---");
        gopay.bayar(60000);
        gopay.bayar(50000);

        System.out.println("\n--- Transaksi Kartu Kredit ---");
        kartuKredit.bayar(450000);
        kartuKredit.bayar(100000);

        System.out.println("\n--- Transaksi Rekening Bank ---");
        mandiri.bayar(150000);
        mandiri.bayar(46000);

        System.out.println("");
        System.out.println(" RIWAYAT TRANSAKSI KESELURUHAN ");
        System.out.println("");

        gopay.getRiwayat();
        kartuKredit.getRiwayat();
        mandiri.getRiwayat();
    }
}
```

- **Hasil Running Program**